



GHCP • GHE

SECURITY ARCHITECTURE

Securing AI-Assisted Development

Deterministic and probabilistic tools working together across the software development lifecycle.



Mehdi Labadi | Solution Expert — Cloud & AI

What we'll cover

01

Two paradigms

Deterministic vs. probabilistic — and how they complement each other.

02

Custom agents + MCP

Building deterministic security tooling on top of GHCP.

03

GHE built-in controls

CodeQL, Dependabot, Secret Scanning, and Push Protection — each with a live demo.

04

Bringing it together

How all of this layers across the SDLC.

Two ways a tool can think



Deterministic

Rules, scripts, scanners, policy engines, MCP servers, and custom agents with fixed logic.

- Same input → same output, always
- Auditable and explainable
- Fast, cheap, and repeatable
- Only catches what was coded for



Probabilistic

LLMs and AI-powered review, triage, and code generation inside Copilot.

- Handles novel, unstructured context
- Generalizes across languages and frameworks
- Non-deterministic — output varies
- Needs guardrails for safety and compliance



PART ONE

Probabilistic Tooling

A custom GHCP-powered security agent that reasons over your code and orchestrates deterministic MCP servers.

Custom security agent



What it is

A scoped Copilot agent with an LLM brain, a curated toolset, a system prompt, and a policy — invoked on demand or automatically in CI.

Guardrails around the AI

- Fixed toolset — no surprise actions
- Versioned prompt and policy
- Runs in a trusted boundary
- Every action is logged and reviewable

Typical capabilities



Scan diffs

Run SAST / secret / IaC checks on each PR



Enforce policy

Block merges violating org security rules



Gate releases

Verify branch protection + signed commits

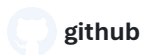


Triage alerts

Assign, de-duplicate, and explain findings

Security-focused MCP servers the agent calls

Hosted MCPs wired into VS Code — no PATs, no keys. One local MCP (Playwright) drives a browser for red-team work.



github

HTTP

Hosted. Auth via Copilot sign-in. Reads code scanning, Dependabot, secret scanning, branch protection; opens issues + PRs.



microsoft-learn

HTTP

Hosted, no auth. Authoritative Microsoft security guidance, Zero Trust patterns, Azure Policy + Defender docs.



deepwiki

HTTP

Hosted, no auth. Repo-level knowledge base — pulls CodeQL, Semgrep, Trivy, and Snyk rules documentation.



context7

HTTP

Hosted, no auth. Up-to-date library docs — look up secure APIs for React, Express, jsonwebtoken, axios, and more.



owasp-cheatsheets

HTTP

Hosted via gitmcp.io. OWASP Cheat Sheet Series — XSS, SQLi, JWT, CORS, authN/Z, secrets, supply chain.



github-advisories

HTTP

Hosted via gitmcp.io. Deterministic CVE / GHSA lookup — real advisory IDs and fixed-version ranges.



playwright

STUDIO

Local via npx. Drives a real browser for red-team runs — XSS payloads, IDOR probes, auth bypass against the demo app.

6 hosted · 1 local

All HTTP MCPs auth via VS Code. Playwright runs locally so it can drive a real browser.

What the agent can take off your plate



Secure pull-request review

Agent calls scanning + policy MCP on every PR, attaches findings as structured review comments.



Secret & credential hygiene

Agent opens an incident on a secret leak and walks the developer through rotation.



Dependency & SBOM triage

SBOM MCP answers 'what uses package X?' deterministically. Agent drafts remediation plan.



Release & compliance gates

Policy MCP validates branch protection, signed commits, and environment reviewers before release.

Two custom agents in action

The probabilistic layer we just described — running against a real app.



DEMO 1

Red-team agent

Authenticated intrusion testing against the running demo app.

- Drives a real browser via Playwright MCP
- XSS, IDOR, prototype pollution, auth bypass probes
- Target: `http://localhost:5173` (isolated session)
- Findings logged and handed to the review workflow

MCP: playwright



DEMO 2

Security review workflow

Agentic PR review with citations and real advisory IDs.

- Pulls findings from code scanning + Dependabot
- Cites OWASP cheat sheets for each remediation
- Attaches real GHSA IDs from the Advisory DB
- Files GitHub issues with repro, patch, and labels

MCPs: github · deepwiki · owasp-cheatsheets · github-advisories

PART TWO

GHE Built-in Deterministic Tools

CodeQL · Dependabot · Secret Scanning · Push Protection — four layers baked into GitHub Enterprise.



CodeQL — static analysis that queries your code



What it does

Finds vulnerabilities (SQLi, XSS, auth flaws, unsafe deserialization, etc.) by running semantic queries over your source code.

Why it's deterministic

- Same query + same code → same result
- Hundreds of curated, versioned queries
- Custom queries for your org's patterns

How it works

1

Build a database

Compile your code into a relational model — functions, calls, data flow.

2

Run queries

CodeQL queries (SQL-like) look for vulnerable patterns and tainted data paths.

3

Report findings

Results surface as code-scanning alerts on PRs with remediation guidance.

Dependabot — dependencies on autopilot



What it does

Continuously checks your direct and transitive dependencies against the GitHub Advisory Database and opens PRs to remediate.

Why it's deterministic

- Manifest + advisory DB = known answer
- Version bumps are reproducible
- Auto-opens PRs on schedule



Alerts

Notifies when a new CVE affects your dependencies.



Security updates

Opens PRs that bump vulnerable versions automatically.



Version updates

Keeps packages current on a schedule you configure.

Secret scanning — find leaked credentials in your code



What it does

Scans the entire git history of every repo for 200+ known secret patterns — API keys, tokens, private keys, database strings.

Partner program

Cloud providers (AWS, Azure, GCP, Stripe, and others) are notified when their tokens leak — many revoke automatically.

What gets scanned



Full git history

Every branch, every commit — not just HEAD



Issues, PRs, wikis

Secrets often hide in descriptions and comments



Custom patterns

Define org-specific tokens with regex



Alerts dashboard

Security team triages, assigns, and tracks rotation

Push protection — stop secrets before they land



What it does

Blocks pushes that contain secrets before they reach the remote — turning remediation from post-incident into pre-commit.

Shift-left, deterministically

- Works in IDE, CLI, and web
- Developer can bypass only with justification
- Every bypass is audit-logged

Without → With push protection



Push → secret scanning alert → incident → rotation

Secret is already in history. Assume compromised.



Push blocked at the client — nothing lands in the repo

Developer fixes locally. No rotation, no incident.

LIVE DEMO

GHE built-in tools in action

Four short walkthroughs — one per tool we just covered.



CodeQL



Run a CodeQL query against a sample repo and walk through an alert on a pull request.



Dependabot



Trigger an alert on a vulnerable package, then merge the auto-generated fix PR.



Secret scanning



Plant a test token, commit it, and watch the alert fire with a partner revocation link.



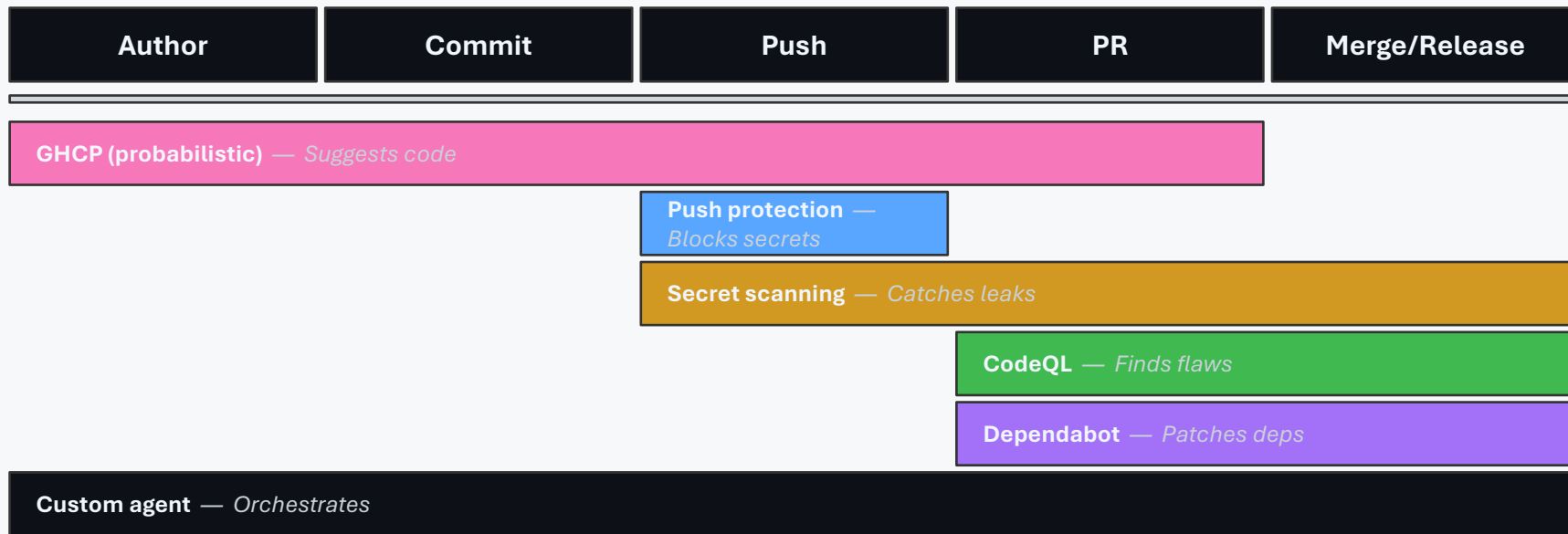
Push protection



Attempt to push a real-looking secret and see the push rejected at the client.

Same story each time: deterministic tool catches the issue — developer fixes with confidence.

Layered across the SDLC



Net effect: every stage of the SDLC has an AI helper and a deterministic enforcer — together, not instead of each other.

KEY TAKEAWAYS

Ship fast. Stay safe.



AI gives you speed

Probabilistic tools generalize and handle the messy cases.



Determinism gives you trust

CodeQL, Dependabot, secret scanning, push protection, and custom agents make behavior auditable.



Combine them on purpose

Let AI propose. Let deterministic tools decide what actually happens across the whole SDLC.

Thank you